

(20 pts Questão 1) Escreva uma **função** que faça o tratamento da interrupção de recepção de dados da serial. Esta função será responsável por ler o valor recebido da serial no registro RCREG e armazená-lo em uma variável global. A flag responsável por indicar que chegou uma informação pela serial é o bit 3 do registro PIR1.

(25 pts Questão 2) Crie um **programa** que leia um sensor de pressão que está conectado na entrada analógica e apresente o seu valor nos 3 primeiros dígitos do display de 7 segmentos. Para evitar o flicker o loop principal deve ser executado na velocidade de 4ms. O último dígito do display de 7 segmentos deve mostrar a letra 'b'. Quando o sensor está sob a ação de uma pressão de 0 bar o conversor analógico lê 0 volts (0 no AD). Sob a pressão de 256 bar a saída fica saturada em 5volts (1024 no AD). O sensor possui uma resposta linear. Utilize as bibliotecas “display.h”, “timer.h” e “adc.h”.

(25 pts Questão 3) Um periférico de conversão analógico-digital (ADC) possui dois conversores: ADC1 e ADC2, conectados nos pinos 4 e 5 da porta A. O comando para iniciar a conversão é feito ligando-se o bit 4 do registro ADCxCONF, (ADC1CONF para o ADC1 e ADC2CONF para o ADC2). Quando o conversor termina a conversão o respectivo bit é desligado. O resultado da conversão é armazenado nos registros ADCxRES. Entre as opções de configuração do ADC, ele permite realizar a conversão em 1, 2, 4 ou 8 milissegundos; utilizar 7 ou 8 bits de resolução; realizar conversões sucessivas. O periférico do ADC é ligado através do bit 5 do ADCxCONF. Estas opções são configuradas através do registro ADCxCONF conforme tabela a seguir:

ADCxCONF (x=1->ADC1; x=2->ADC2)		
Bits	Desligado	Ligado
0	7 bits	8 bits
1	Conversões sucessivas	Conversão normal
2-3	00-1ms; 01-2ms; 10-4ms; 11-8ms	
4	Fim de conversão	Inicia conversão/convertendo
5	ADC ligado	ADC desligado
6-7	Não utilizado	

Crie uma **biblioteca** (arquivo .c e .h) com três funções: uma para inicializar o periférico do ADC e as outras duas para iniciar, aguardar a conversão e devolver o resultado de cada um dos conversores. Lembrar de configurar o terminal como entrada.

(30 pts Questão 4) Um liquidificador pode ser ligado com diferentes velocidades. Um modo de se implementar o controle de velocidade do motor é utilizar uma saída PWM conectada à um motor DC, variando-se o tamanho do pulso do PWM. Crie um programa que implementa um liquidificador com 4 velocidades (25, 50, 75 e 100%) e um modo de ativação pulsado (função pulsar, onde o motor fica 1 segundo ligado em 100% e 1 segundo desligado em 0%). A placa conta com 6 botões, um para desligar, quatro para as diferentes velocidades e um para a função pulsar. Os botões estão conectados diretamente no microcontrolador através dos bits 0 à 6 da porta B. O motor está conectado no PWM 1. Faça um **programa** que controle o sistema proposto. Lembre-se de implementar uma rotina de debounce para os botões utilizados. Utilize as bibliotecas timer.h e pwm.h

Funções disponíveis nas bibliotecas (usar de acordo com o enunciado dos problemas).

```
//adc.h
void adcInit(void);
unsigned int adc1Read(void);
unsigned int adc2Read(void);
```

```
//display.h
void ssdInit(void);
void ssdUpdate(void);
void ssdDigit(char valor, char posicao);
```

```
//timer.h
void timerInit(void);
void timerReset(int tempo); //tempo em us
void timerWait(void);
```

```
//pwm.h
void InicializaPWM(void);
void SetaFreqPWM(int freq);
void SetaPWM1(int porcentagem);
```

RESPOSTA EX 1

5 pt pra criar a variável global
5 pt pra lembrar do interrupt 1
5 pt pra testar certo o bit de flag e limpar a flag depois
5 pt pra ler o valor/armazenar na variável

```
static int varSerialGlobal;
void ISR (void) interrupt 1 { //o nome da função não importa
    if (BitTst(PIR1,3)){ // teste para saber se a flag do AD está ligada
        varSerialGlobal = RCREG;
        BitClr(PIR1,3);
    }
}
```

RESPOSTA EX 2

5 pt inicialização dos 3 sistemas.
5 pt pra fazer o loop temporizado corretamente
10 pt por ler o valor da pressão e converter corretamente
5 pt por separar os algarismos corretamente
(se usar um loop for pra contar tempo tira uns 5 pontos)

```
#include "config.h"
#include "timer.h"
#include "disp7seg.h"
#include "adc.h"
void main (void){
    int pressao;
    ssdInit();
    adcInit();
    timerInit();
    for(;;){
        timerReset(4000); //4ms
        pressao = adcRead();
        pressao /= 4;
        ssdDigit((pressao/100)%10, 3);
        ssdDigit((pressao/10)%10, 2);
        ssdDigit((pressao/1)%10, 1);
        ssdDigit(11, 0); //11 = b em hexadecimal
        timerWait(); //Tempo pra evitar flicker
    }
}
```

RESPOSTA EX 3

5 pt criação do header
10 pt inicialização correta (TRIS e ADxCONF)
10 pt para as funções de leitura (wait + return)

```
//----- ADC.H
#ifndef ADC_H
#define ADC_H
void adcInit(void);
int adcRead1(void);
int adcRead2(void);
#endif
//----- ADC.C
#include "adc.h"
#include "pic18f4520.h"
void adcInit(void){
    BitSet(TRISA,4); //seta o bit 4 como entrada - ADC1
    BitSet(TRISA,5); //seta o bit 5 como entrada - ADC2
    AD1CONF = 0b00000010; //ADC1 ligado, conversão normal, o resto é opcional
    AD2CONF = 0b00000010; //ADC2 ligado, conversão normal, o resto é opcional
}
int adcRead1(void){
    BitSet(AD1CONF,4); //inicia conversao
```

```

        while(BitTst(AD1CONF,4)); // espera terminar a conversão;
        return AD1RES;// le o resultado
    }
    int adcRead2(void){
        BitSet(AD2CONF,4); //inicia conversao
        while(BitTst(AD2CONF,4)); // espera terminar a conversão;
        return AD1RES;// le o resultado
    }

```

RESPOSTA EX 4

5 pt inicialização dos sistemas e das chaves

10 pt para a rotina de debounce correta

10 pt testes das chaves comuns

10 pt para implementação do ciclo temporizado de pulso

```

#include "pic18f4520.h"
#include "config.h"
#include "pwm.h"
#include "timer.h"

void main (void){
    int pulsar;

    int tempoPulso;

    int chaves;
    int chavesTemp;

    int debounce = 10;

    TRSIB = 0x3F; // seis primeiros bits como entrada
    pwmInit();
    timerInit();
    pwmFrequency(10000); //é importante configurar, o valor não é crítico
    for(;;){
        timerReset(1000); // loop de 1ms

        //teve mudança?
        if(PORTB != chavesTemp){
            //reseta contagem do debounce
            debounce = 10;
            chavesTemp = PORTB;
        }
        //se estabilizar diminui o tempo
        if(chavesTemp == PORTB){
            debounce--;
        }
        //chegou ao fim do tempo atualiza a variável
        if(debounce == 0){
            chaves = chavesTemp;
        }

        //todos os testes são realizados na variavel depois do debounce
        if(bitTst(chave,0)){
            pwm1Set(0);
            pulsar = 0;
        }
        if(bitTst(chave,1)){
            pwm1Set(25);
        }
        if(bitTst(chave,2)){
            pwm1Set(50);
        }
        if(bitTst(chave,3)){
            pwm1Set(75);
        }
    }
}

```

```
if(bitTst(chave,4)){
    pwm1Set(100);
}

if(bitTst(chave,5)){
    pulsar = 1;
}

//se a opção pulsar estiver ativa, tem que ficar 1000 ms ligado e 1000 ms desligado.
if(pulsar){
    tempoPulso++;
    if((tempoPulso>0)&&(tempoPulso<1000)){
        pwm1Set(0);
    }
    if((tempoPulso>1000)&&(tempoPulso<2000)){
        pwm1Set(100);
    }
    if(tempoPulso>2000){
        tempoPulso = 0;
    }
}
timerWait();
}
```